GUIDA INDIVIDUALE / PREPARAZIONE ALL'ORALE

Il progetto, spiegato con il codice

Una lettura guidata per capire che cosa succede, perché succede e come raccontarlo al professore.

Marco Pernisco 683674 / P2P Systems and Blockchains Lab

Da dove partiamo

Immagina una cassa comune. Alcune persone vi depositano ETH; un'altra persona chiede un prestito. Chi ha messo i fondi vota se concederlo. Se il prestito viene approvato, il denaro esce dalla cassa e arriva al richiedente. Quando rientra, una parte restituisce il capitale e un'altra paga gli interessi.

Il nostro progetto traduce questa storia in contratti Solidity e programmi Python. La parte difficile non è ricordare i nomi: è capire come ogni istruzione cambia i soldi disponibili, i crediti e lo stato di un prestito.

Come studiare ogni estratto

Leggi prima la situazione descritta. Poi guarda il codice e chiediti: **chi lo chiama, che cosa controlla, quali dati cambia e a chi invia ETH?** Infine prova a rispondere alla domanda nel riquadro senza leggere la soluzione. La risposta deve spiegare il motivo del codice, non tradurre soltanto le parole.

Tre percorsi di lettura

Il prestito	Persone e saldi; depositi; voto; erogazione; rate; default e compensazione.
I programmi	Oracle Bitcoin, saldi UTXO, attesa iniziale, voter e gestione degli eventi.
La discussione	Setup, proxy, sicurezza, test, gas, strategia economica e demo.

Codice vero, non pseudocodice

Gli estratti sono letti direttamente dai file del progetto durante la compilazione del PDF. I numeri a sinistra sono le righe dei sorgenti. Mostriamo porzioni di funzioni: il contesto e gli altri controlli sono spiegati nel testo. Se i file cambiano, i riferimenti vanno ricontrollati. La guida è separata dal report di consegna e non modifica le regole dei contratti.

01 / LA STORIA COMPLETA

Chi mette i fondi e chi li riceve

Prima distinguiamo le persone dai programmi che usano.

Chiamiamo **Alice** la persona che deposita nel pool e **Bob** quella che chiede il prestito. Questi nomi servono solo agli esempi; il codice identifica le persone con indirizzi Ethereum.

Alice è un *contributor*: mette ETH a disposizione, vota e può ritirare la parte non impegnata in prestiti. Bob è l'*applicant*, cioè il richiedente; quando riceve il prestito lo chiamiamo anche *borrower*. Una stessa persona può ricoprire entrambi i ruoli. Esistono poi l'owner, che amministra il pool, e l'operatore oracle, che pubblica il dato Bitcoin.

La sequenza, raccontata senza formule

Bob indica un indirizzo Bitcoin e paga una richiesta di aggiornamento. Un programma Python ricostruisce quanto quell'indirizzo possiede nello snapshot storico richiesto e scrive il risultato su Ethereum. Bob propone il prestito; Alice vota. Passati i blocchi previsti, Bob chiede al pool di risolvere la proposta.

Se voto, disponibilità ETH e controllo BTC permettono il prestito, il pool crea un contratto dedicato e gli passa il capitale. Il contratto lo invia a Bob. Bob paga le rate a quel contratto, che comunica al pool come dividerle. Se Bob non paga in tempo, il prestito può essere dichiarato fallito. Alice può chiedere una compensazione, ma solo nei limiti della riserva realmente esistente.

Dove cercare i vari passaggi

File	In parole semplici
`FundingPool.sol`	La cassa comune e i suoi registri: depositi, voti e riparti.
`LoanContract.sol`	La scheda di un singolo prestito, con termini e scadenza.
`BitcoinOracle.sol`	Il registro Ethereum dei saldi BTC comunicati dall'operatore.
`FundingPoolProxy.sol`	L'indirizzo stabile del pool, anche quando cambia la logica.
`oracle_service.py`	Legge Bitcoin e risponde alle richieste di saldo.
`auto_voter.py`	Vota sempre YES con l'account configurato.
`simulate.py`	Fa compiere agli account la sequenza della dimostrazione.

Il professore potrebbe chiedere: perché non fa tutto un unico contratto?

Il pool gestisce i fondi comuni; ogni contratto di prestito gestisce una singola operazione. Il dato Bitcoin richiede invece un servizio esterno, perché il contratto Ethereum non apre da solo i file Bitcoin del computer. Separare questi compiti rende più chiaro chi può modificare ciascun dato.

02 / LEGGERE SOLIDITY

Le parole che trovi nel codice

Pochi concetti di sintassi per non bloccarti davanti a una funzione.

contracts/FundingPool.sol / righe 70–75

```
70      mapping(address => uint256) public totalContributed;
71      mapping(address => uint256) public lockedContributions;
72      mapping(address => uint256) public pendingInterestGain;
73      mapping(uint256 => LoanProposal) public proposals;
74      mapping(uint256 => mapping(address => bool)) public hasVoted;
75      mapping(uint256 => mapping(address => bool)) public voteSupport;
```

Un `mapping` è un registro indicizzato da una chiave. Nella riga 70 la chiave è un indirizzo: per ogni persona salvo un importo. Nella riga 74 ci sono due chiavi: prima l'identificativo della proposta, poi l'indirizzo del votante. Così posso chiedere: "Alice ha già votato sulla proposta numero 2?".

`address` rappresenta un indirizzo Ethereum; `uint256` è un intero senza segno; `bool` vale vero o falso. Gli importi ETH sono interi in **wei**: 1 ETH corrisponde a 10^{18} wei. Il contratto non usa numeri con la virgola per i pagamenti.

contracts/FundingPool.sol / righe 141–148

```
141     function contribute() external payable notTerminated {
142         if (msg.value <= MIN_DEPOSIT_WEI) revert DepositTooLow();
143
144         totalContributed[msg.sender] += msg.value;
145         totalUnlockedFunds += msg.value;
146         _registerContributor(msg.sender);
147
148         emit ContributionAdded(msg.sender, msg.value);
```

Qui `external` permette di chiamare la funzione dall'esterno. `payable` permette di accompagnare la chiamata con ETH. `msg.sender` è chi sta chiamando il contratto, mentre `msg.value` è quanto ETH accompagna questa chiamata, espresso in wei.

`notTerminated` è un *modifier*: esegue un controllo aggiuntivo prima del corpo della funzione. `revert` interrompe l'operazione e annulla gli effetti della transazione; il gas già consumato non viene magicamente azzerato. `emit` produce un evento nei log: è una notifica che Python può leggere, non una chiamata diretta al programma Python. Nel corpo di un modifier, `_` segna il punto in cui viene eseguita la funzione protetta.

Altre parole che incontreremo

`storage` indica dati persistenti del contratto: un riferimento storage permette di modificarli direttamente. `memory` indica dati temporanei della chiamata. `view` indica che la funzione non modifica lo stato; `pure` che non legge né modifica lo stato del contratto. `immutable` fissa un dato alla costruzione del contratto. `internal` serve alla logica interna o ereditata, non è un normale ingresso pubblico per l'utente.

> **Il professore potrebbe chiedere: public significa che tutti possono modificare il saldo?**
>
> No. Per questo mapping il compilatore genera una funzione di lettura. Le modifiche devono passare per il codice delle funzioni che le permettono. Visibilità del dato e autorizzazione a cambiarlo sono due cose diverse.

03 / I TRE SALDI

Depositato non vuol dire disponibile

Il punto più importante della contabilità del progetto.

Alice deposita 10 ETH. Finché non presta nulla, può ritirare 10. Se 4 vengono erogati a Bob, Alice ha ancora una posizione complessiva di 10, ma può ritirarne solo 6: gli altri 4 sono impegnati nel prestito.

contracts/FundingPool.sol / righe 172–174

```
172     function unlockedBalanceOf(address contributor) public view returns (uint256) {
173         return totalContributed[contributor] - lockedContributions[contributor];
174     }
```

La funzione legge il totale registrato per Alice e sottrae il capitale bloccato. Non interroga il saldo del wallet di Alice e non trasferisce ETH. Calcola la parte della sua posizione che il pool può ancora utilizzare o restituirle.

Situazione (ETH)	Totale	Bloccato	Disponibile
Alice deposita	10	0	10
Bob riceve un prestito di 4	10	4	6
Bob restituisce 2 di capitale	10	2	8
Bob restituisce gli ultimi 2	10	0	10

Perché il totale non sale quando Bob rimborsa?

Perché quei 4 ETH erano già parte della posizione di Alice. Il rimborso cambia la loro condizione: da impegnati tornano disponibili. Se aggiungessi anche 4 al totale, dopo il rimborso Alice risulterebbe creditrice di 14 ETH pur avendone depositati 10, senza nemmeno contare gli interessi. Sarebbe un doppio conteggio.

Nel pool esistono anche due somme separate: il **gain**, cioè gli interessi da incassare, e la **riserva di compensazione**, usata per le perdite. Nessuna delle due aumenta automaticamente il capitale disponibile per nuovi prestiti. Il saldo ETH del contratto e il capitale totale registrato per le persone, quindi, non sono la stessa misura.

Una regola da ricordare con un esempio, non solo come formula

Totale = disponibile + bloccato. Nel nostro esempio: $10 = 6 + 4$. Nei normali flussi, gli ETH effettivamente nel pool coprono capitale disponibile, riserva e gain non incassati. Il capitale bloccato è già stato erogato: non è ancora denaro liquido dentro il contratto.

Il professore potrebbe chiedere: perché non posso ritirare anche il bloccato?

Perché il pool lo ha già prestato. La variabile rappresenta quanto resta da recuperare, non ETH ancora disponibili in cassa. Per ritirarlo deve prima rientrare il capitale, oppure deve avvenire un recupero secondo le regole della compensazione.

04 / DEPOSITI E PRELIEVI

Come entrano ed escono gli ETH

Seguire l'ordine: controlli, aggiornamenti e trasferimento.

contracts/FundingPool.sol / righe 141–148

```
141     function contribute() external payable notTerminated {
142         if (msg.value <= MIN_DEPOSIT_WEI) revert DepositTooLow();
143 
144         totalContributed[msg.sender] += msg.value;
145         totalUnlockedFunds += msg.value;
146         _registerContributor(msg.sender);
147 
148         emit ContributionAdded(msg.sender, msg.value);
```

Il deposito usa l'importo della chiamata, non un parametro dichiarato a parole. La riga 142 rifiuta un importo minore o *uguale* al minimo: servono **più di 100.000 wei**. Poi aggiorna il totale della persona e la disponibilità del pool. La registrazione aggiunge la persona all'elenco dei contributor solo se non c'è già; l'evento permette ai client di osservare il deposito.

contracts/FundingPool.sol / righe 153–166

```
153     function withdraw(uint256 amount) external virtual {
154         if (amount == 0) revert AmountMustBeGreaterThanZero();
155 
156         uint256 unlocked = unlockedBalanceOf(msg.sender);
157         if (amount > unlocked) revert InsufficientUnlockedBalance();
158 
159         totalContributed[msg.sender] -= amount;
160         totalUnlockedFunds -= amount;
161         _unregisterEmptyContributor(msg.sender);
162 
163         (bool sent, ) = payable(msg.sender).call{value: amount}("");
164         if (!sent) revert TransferFailed();
165 
166         emit ContributionWithdrawn(msg.sender, amount);
```

Il prelievo prima verifica che l'importo sia positivo e non superi il disponibile. Poi riduce i registri, e soltanto dopo invia gli ETH. Con 10 totali e 4 bloccati, Alice può prelevare al massimo 6. Se preleva 2, rimangono 8 totali, 4 bloccati e 4 disponibili.

Nella riga 163 `payable(msg.sender)` indica il destinatario; la `call` invia `amount` wei. Il booleano `sent` dice se la chiamata è riuscita. Se fallisce, il `revert` annulla anche le modifiche ai registri: Alice non perde il credito senza ricevere il denaro.

> **Il professore potrebbe chiedere: perché aggiorni i saldi prima della call?**
>
> Perché il destinatario potrebbe essere un contratto ed eseguire altro codice appena riceve ETH. Aggiornando prima, una chiamata ricorsiva non vede più lo stesso saldo disponibile da prelevare. È la difesa principale mostrata nel nostro test di reentrancy.

05 / CREARE E VOTARE

La proposta non è ancora il prestito

Salvare i termini e registrare una scelta sono operazioni diverse dall'erogazione.

Bob deve chiedere un prestito a proprio nome, indicare un indirizzo BTC non vuoto, un capitale positivo, un tasso tra 1 e 100 e una durata positiva. Nel codice il parametro del voto deve essere **esattamente 12**: la risoluzione è possibile dopo che sono trascorsi quei 12 blocchi.

contracts/FundingPool.sol / righe 460–474

```
460         if (borrower == address(0)) revert InvalidAddress();
461         if (borrower != msg.sender) revert Unauthorized();
462         if (bytes(borrowerBtcAddress).length == 0) revert InvalidBitcoinAddress();
463         if (principal == 0 || durationBlocks == 0) revert AmountMustBeGreaterThanZero();
464         if (votingDurationBlocks != PROPOSAL_VOTING_PERIOD_BLOCKS) revert InvalidVotingPeriod();
465         if (interestRate < 1 || interestRate > 100) revert InvalidInterestRate();
466
467         proposalId = ++proposalCount;
468         LoanProposal storage proposal = proposals[proposalId];
469         proposal.borrower = borrower;
470         proposal.borrowerBtcAddress = borrowerBtcAddress;
471         proposal.principal = principal;
472         proposal.interestRate = interestRate;
473         proposal.durationBlocks = durationBlocks;
474         proposal.votingDeadlineBlock = block.number + PROPOSAL_VOTING_PERIOD_BLOCKS;
```

La riga 467 assegna un nuovo numero alla proposta. La riga 468 prende un riferimento `storage`: le assegnazioni successive modificano la proposta salvata nel contratto, non una copia temporanea. La scadenza del voto è espressa come altezza di blocco, non come data o secondi dell'orologio.

contracts/FundingPool.sol / righe 490–502

```
490     function voteOnProposal(uint256 proposalId, bool support) external notTerminated {
491         LoanProposal storage proposal = proposals[proposalId];
492         if (proposal.borrower == address(0)) revert InvalidProposal();
493         if (proposal.executed) revert VotingClosed();
494         if (hasVoted[proposalId][msg.sender]) revert AlreadyVoted();
495
496         // Eligibility depends on the entire position, including locked value.
497         // A zero-weight vote can become relevant before resolution.
498         if (totalContributed[msg.sender] == 0) revert NoVotingPower();
499         uint256 weight = unlockedBalanceOf(msg.sender);
500
501         hasVoted[proposalId][msg.sender] = true;
502         voteSupport[proposalId][msg.sender] = support;
```

Si rifiutano proposte inesistenti, già risolte e voti duplicati. Il controllo alla riga 498 usa il *totale*, quindi un contributor con tutto il capitale bloccato può votare. Il suo peso corrente può però essere zero. La coppia di mapping salva separatamente "ha votato" e "ha scelto YES o NO".

> **Il professore potrebbe chiedere: perché servono due mapping e non basta voteSupport?**
>
> Un booleano non impostato vale già false. Senza `hasVoted` non distinguerei chi ha scelto NO da chi non ha ancora votato, né potrei impedire correttamente un secondo voto. Per decidere l'esito entrambi pesano NO, ma sono situazioni diverse.

06 / DECIDERE L'ESITO

Il voto viene ripesato alla fine

La scelta resta la stessa, ma il capitale disponibile può cambiare.

Immagina Alice con 6 ETH disponibili e Carlo con 4. Alice vota YES e Carlo non vota: in quel momento Alice avrebbe la maggioranza. Prima della risoluzione, però, Alice ritira 4 ETH. Le disponibilità diventano 2 e 4: il suo YES non ha più la maggioranza. Il codice deve guardare la situazione corrente.

contracts/FundingPool.sol / righe 524–538

```
524         uint256 currentDisposable = totalUnlockedFunds;
525         uint256 currentYesVotes = 0;
526         for (uint256 i = 0; i < contributors.length; i++) {
527             address contributor = contributors[i];
528             if (hasVoted[proposalId][contributor] && voteSupport[proposalId][contributor]) {
529                 currentYesVotes += unlockedBalanceOf(contributor);
530             }
531         }
532         proposal.yesVotes = currentYesVotes;
533         proposal.noVotes = currentDisposable - currentYesVotes;
534 
535         if (currentYesVotes <= (currentDisposable / 2)) {
536             proposal.executed = true;
537             emit LoanRejected(proposalId, "Majority Reject");
538             return address(0);
```

Il ciclo somma soltanto la disponibilità corrente di chi ha votato YES. La riga 533 considera NO tutto il resto: sia i contrari, sia chi non ha votato. Il controllo alla riga 535 richiede più di metà del capitale disponibile. Una parità non basta. Anche quando la somma è dispari, il confronto intero funziona: su 5 wei totali, 3 YES bastano e 2 no.

Perché non uso solo i pesi registrati al momento del voto?

`voteOnProposal()` aggiorna anche i conteggi della proposta, ma quei valori non sono il risultato definitivo: qui vengono ricalcolati e sovrascritti. Nel frattempo possono esserci stati depositi, prelievi e rimborsi di altri prestiti. La regola del progetto richiede i saldi disponibili alla risoluzione.

Nel codice, `votingDeadlineBlock` indica quando Bob può risolvere: dal blocco di creazione più 12 in poi. Non chiude il voto da solo: si può votare finché la proposta non viene risolta. Solo Bob può risolverla. Una risoluzione anticipata fallisce; il rifiuto economico invece chiude la proposta, emette un evento e restituisce l'indirizzo zero.

> **Il professore potrebbe chiedere: a cosa serve executed = true anche quando rifiuti?**
>
> A rendere definitiva quella risoluzione. Il nome del campo non significa necessariamente "denaro erogato": significa proposta già trattata. Senza chiuderla, Bob potrebbe provare ripetutamente a farla passare dopo aver cambiato la situazione.

07 / TROVARE IL CAPITALE

Da quali persone arriva il prestito

Prima verifico la capacità; poi blocco le quote proporzionali.

contracts/FundingPool.sol / righe 547–553

```
547         uint256 borrowerBalanceSats = bitcoinOracle.getBalance(proposal.borrowerBtcAddress);
548         uint256 borrowerLiquidityInWei = (borrowerBalanceSats * BTC_TO_ETH_RATE * 1 ether) / SATS_PER_BTC;
549         if (borrowerLiquidityInWei < proposal.principal) {
550             proposal.executed = true;
551             emit LoanRejected(proposalId, "Bitcoin Liquidity Check Failed");
552             return address(0);
553         }
```

L'oracle restituisce satoshi. Il pool li converte in wei con il cambio fisso di **30 ETH per BTC**. `1 ether` è una costante di unità di Solidity: vale 10^{18} wei, non una richiesta di inviare un ETH. Con 0,04 BTC la capacità è 1,2 ETH: una richiesta di 1 può passare questo controllo, una di 2 no. Devono comunque essere sufficienti anche i fondi ETH del pool.

contracts/FundingPool.sol / righe 262–273

```
262             uint256 lockAmount = Math.mulDiv(disposable, requiredAmount, totalDisposable);
263             if (lockAmount == 0) continue;
264 
265             lockedContributions[contributor] += lockAmount;
266             totalUnlockedFunds -= lockAmount;
267 
268             loanContributions[proposalId][contributor] = lockAmount;
269             initialLoanContributions[proposalId][contributor] = lockAmount;
270             loanContributors[proposalId].push(contributor);
271             emit FundsLocked(contributor, lockAmount);
272 
273             totalLocked += lockAmount;
```

Queste righe sono dentro il ciclo sui contributor. Se il pool ha 60 ETH disponibili, divisi in 10, 20 e 30, per un prestito di 12 le quote sono 2, 4 e 6. `Math.mulDiv` calcola il prodotto diviso per il totale e, in questa forma, arrotonda per difetto. Tutti i contributor disponibili partecipano, non solo chi ha votato YES.

La quota aumenta il bloccato della persona e diminuisce la disponibilità comune. Vengono salvate due copie con significati diversi: `loanContributions` è il credito che si ridurrà con i recuperi; `initialLoanContributions` conserva la quota iniziale per ordine di rimborso e interessi.

> **Il professore potrebbe chiedere: perché sommi totalLocked invece di erogare la richiesta?**
>
> Perché le quote intere possono perdere un resto. Con disponibilità 100.001 e 100.002 wei e richiesta di 3, ogni quota è 1: erogo 2, non 3. I controlli di capacità vengono prima effettuati sulla richiesta; il contratto del prestito usa poi il capitale effettivo, che può essere inferiore per arrotondamento.

08 / IL CONTRATTO DEL PRESTITO

Una scheda separata per Bob

Il pool crea il contratto e gli dà i fondi per l'erogazione.

contracts/FundingPool.sol / righe 563–575

```
563         LoanContract loan = new LoanContract(
564             address(this),
565             proposal.borrower,
566             disbursedAmount,
567             proposal.interestRate,
568             proposal.durationBlocks,
569             collateralPercentage
570         );
571
572         authorizedLoanContracts[address(loan)] = true;
573         loanToProposalId[address(loan)] = proposalId;
574         activeLoanCount += 1;
575         loan.fundLoan{value: disbursedAmount}();
```

`new LoanContract` distribuisce una nuova istanza su Ethereum. Le passo indirizzo del pool, borrower, capitale effettivo, tasso, durata e percentuale di collateral corrente. Ogni prestito ha quindi i propri termini. Il pool registra l'indirizzo come autorizzato prima di finanziarlo.

contracts/LoanContract.sol / righe 91–98

```
91      function fundLoan() external payable onlyPool {
92          if (status != LoanStatus.Created) revert InvalidState();
93          if (msg.value != principal) revert InvalidAmount();
94
95          status = LoanStatus.Funded;
96
97          (bool sent, ) = payable(borrower).call{value: msg.value}("");
98          if (!sent) revert TransferFailed();
```

`onlyPool` permette il finanziamento solo al pool associato. Il prestito deve essere nello stato Created e ricevere esattamente il capitale previsto. Passa a Funded e invia gli ETH a Bob. Se l'invio fallisce, la transazione si annulla: non resta un finanziamento registrato senza l'erogazione riuscita.

Qui msg.sender cambia lungo la catena delle chiamate

Bob chiama il pool per risolvere: nel pool il chiamante è Bob. Il pool chiama `fundLoan()`: dentro il prestito il chiamante è il pool, non Bob. La variabile `borrower` serve a ricordare chi deve ricevere gli ETH e chi potrà rimborsare. `msg.sender` non indica sempre l'utente iniziale.

Il costruttore fissa `dueBlock = block.number + durationBlocks`. Fissa anche il collateral del prestito: se nasce al 50%, continuerà a usare il 50% anche quando il parametro globale del pool cambierà. Gli stati principali sono Created, Funded, Repaid e Defaulted: proposta e prestito sono due oggetti diversi, con momenti diversi del ciclo di vita.

> **Il professore potrebbe chiedere: perché controlli authorizedLoanContracts?**
>
> Per riconoscere i contratti che il pool ha davvero creato. Più avanti il prestito chiamerà funzioni che sbloccano fondi e registrano default: non posso permettere che un indirizzo qualsiasi si presenti come un prestito e alteri quei conti.

09 / CAPIRE UNA RATA

Che cosa paga davvero Bob

Prima facciamo i conti in ETH; poi leggiamo il calcolo Solidity.

Bob riceve 1 ETH al 5%. Deve restituire 1 ETH di capitale e 0,05 di interesse, per un totale di 1,05. Il tasso è sul prestito intero: non stiamo applicando un tasso annuale né un interesse composto.

contracts/LoanContract.sol / righe 79–86

```
79      function dueAmount() public view returns (uint256) {
80          return principal + totalInterestAmount();
81      }
82
83      /// @notice Calculates the total interest amount to be paid on the principal.
84      /// @return Total interest in wei.
85      function totalInterestAmount() public view returns (uint256) {
86          return Math.mulDiv(principal, interestRate, 100);
```

`dueAmount()` restituisce il debito totale previsto, non il residuo dopo le rate già pagate. L'interesse è una divisione intera; il capitale effettivamente erogato è il punto di partenza del calcolo.

Se Bob paga due rate da 0,525 ETH e il collateral del prestito è 50%, ciascuna rata contiene 0,5 di capitale e 0,025 di interesse. Metà dell'interesse va alla riserva e metà è il gain del contributor. Il collateral **non** trattiene il 50% del capitale prestato.

Destinazione	Una rata	Totale due rate
Capitale restituito	0,5000	1,0000
Riserva	0,0125	0,0250
Gain	0,0125	0,0250
Pagamento di Bob	0,5250	1,0500

Quanto della rata accetto come debito?

contracts/LoanContract.sol / righe 117–121

```
117         uint256 expectedInterest = totalInterestAmount();
118         uint256 debt = principal + expectedInterest;
119         uint256 paidBefore = repaidPrincipal + repaidInterest;
120         uint256 accepted = Math.min(msg.value, debt - paidBefore);
121         uint256 paidAfter = paidBefore + accepted;
```

`paidBefore` somma capitale e interessi già contabilizzati. `debt - paidBefore` è il debito residuo. Se Bob deve ancora 0,525 ma manda 0,6, accetto 0,525 come rimborso del debito; gli altri 0,075 sono un'eccedenza e finiranno nella riserva, non vengono restituiti a Bob.

> **Il professore potrebbe chiedere: perché non metti tutti i soldi della rata nel disponibile?**
>
> Perché solo il capitale ricostituisce i fondi prestabili. Gli interessi hanno due destinazioni separate: gain dei contributor e riserva. Sommarli al disponibile falserebbe la contabilità e permetterebbe di prestare somme già dovute ad altro titolo.

10 / IL CALCOLO DELLE RATE

Perché guardo quanto è stato pagato

Non riparto da zero a ogni rata: aggiorno un totale progressivo.

Il problema è l'arrotondamento: dividere separatamente ogni pagamento può produrre risultati diversi a seconda di quante rate Bob sceglie. Per questo calcoliamo prima quale dovrebbe essere il totale raggiunto dopo l'ultimo pagamento, poi sottraiamo quello che avevamo già registrato.

contracts/LoanContract.sol / righe 125–135

```
125         uint256 principalAfter = Math.mulDiv(paidAfter, principal, debt);
126         uint256 interestAfter = paidAfter - principalAfter;
127         uint256 baseAmount = principalAfter - repaidPrincipal;
128         uint256 interestPaid = interestAfter - repaidInterest;
129         uint256 excessAmount = msg.value - accepted;
130         uint256 currentCollateralAmount = Math.mulDiv(interestAfter, collateralPercentage, 100)
131             - Math.mulDiv(repaidInterest, collateralPercentage, 100);
132         uint256 gainAmount = interestPaid - currentCollateralAmount;
133         repaidPrincipal = principalAfter;
134         repaidInterest = interestAfter;
135         currentCollateralAmount += excessAmount;
```

Nella riga 125 `paidAfter` è quanto Bob ha pagato complessivamente, limitato al debito. La proporzione fra capitale e debito ci dice quanto di quel totale è capitale. La riga 126 assegna il resto all'interesse. Le righe 127–128 calcolano solo la parte nuova, che appartiene alla rata attuale.

Con il prestito di 1 ETH, dopo la prima rata abbiamo 0,5 di capitale e 0,025 di interesse. Dopo la seconda, i totali diventano 1 e 0,05. La seconda rata contabilizza le differenze: $1 - 0{,}5 = 0{,}5$ e $0{,}05 - 0{,}025 = 0{,}025$. Non deve distribuire di nuovo i totali già raggiunti con la prima rata.

La stessa idea viene usata per il collateral

Le righe 130–131 calcolano quanto dovrebbe esserci complessivamente nella riserva per gli interessi pagati, e tolgono la parte già conteggiata prima. La riga 132 assegna il resto dell'interesse al gain. Solo dopo aggiorniamo `repaidPrincipal` e `repaidInterest`: i vecchi valori sono serviti per calcolare le differenze. L'eccedenza viene aggiunta alla quota di riserva.

La formula matematica è $B(C) = \lfloor CP/(P+I) \rfloor$: C è il pagato totale, P il capitale e I l'interesse previsto. Non occorre recitarla senza capirla: è una proporzione applicata al totale raggiunto. È una scelta implementativa documentata per rendere coerenti rate e arrotondamenti.

> **Il professore potrebbe chiedere: e se il professore ti indica principalAfter - repaidPrincipal?**
>
> Direi: "Il primo valore è il capitale che risulta rimborsato dopo questa rata; il secondo è quello già rimborsato prima. La differenza è il nuovo capitale da sbloccare adesso. Se distribuissi principalAfter per intero, conterei due volte le rate precedenti."

11 / COLLEGARE I CONTRATTI

Come il rimborso torna al pool

Prima arriva il denaro; poi vengono aggiornati i registri comuni.

Bob chiama `repay()` sul contratto del proprio prestito. Dopo aver calcolato capitale, gain e quota di riserva, il prestito deve trasferire gli ETH al pool e spiegargli come dividerli. Non basta aggiornare i contatori del solo prestito: i saldi dei contributor si trovano nel pool.

contracts/LoanContract.sol / righe 137–145

```
137         if (repaidPrincipal == principal && repaidInterest == expectedInterest && status == LoanStatus.Funded
        ) {
138             status = LoanStatus.Repaid;
139         }
140
141         (bool sent, ) = payable(pool).call{value: msg.value}("");
142         if (!sent) revert TransferFailed();
143
144         IFundingPool(pool).handleRepayment(baseAmount, gainAmount, currentCollateralAmount);
145         emit Repayment(baseAmount, interestPaid, excessAmount);
```

La prima condizione chiude con successo soltanto un prestito ancora Funded che abbia ricevuto tutto il capitale e l'interesse. Se era già Defaulted, quel controllo non lo trasforma in Repaid: è il motivo concreto per cui un pagamento tardivo non cancella il fallimento.

La riga 141 invia al pool tutto il denaro della rata, compresa un'eventuale eccedenza. Il `receive()` del pool accetta l'invio soltanto da prestiti autorizzati. Poi l'interfaccia `IFundingPool` permette di chiamare `handleRepayment()`: non crea un altro pool, descrive come chiamare quello all'indirizzo già noto. In questo callback il chiamante è il prestito.

contracts/FundingPool.sol / righe 398–410

```
398         if (!authorizedLoanContracts[msg.sender]) revert Unauthorized();
399
400         uint256 proposalId = loanToProposalId[msg.sender];
401
402         if (baseAmount > 0) {
403             _refundLockedContributors(proposalId, baseAmount);
404         }
405         if (gainAmount > 0) {
406             _creditProportionalInterestGain(proposalId, gainAmount);
407         }
408         if (collateralAmount > 0) {
409             compensationPoolBalance += collateralAmount;
410             emit CompensationPoolFunded(collateralAmount, LoanContract(msg.sender).collateralPercentage());
```

Il controllo iniziale protegge questi aggiornamenti: un estraneo non può chiamarli inventando capitale da sbloccare. Poi il pool usa tre strade: restituisce il capitale, accredita il gain e aumenta la riserva. I parametri descrivono gli ETH già ricevuti, non inviano tre nuovi pagamenti.

> **Il professore potrebbe chiedere: perché ci sono sia una call con ETH sia handleRepayment?**
>
> La prima sposta il denaro dal prestito al pool; la seconda aggiorna la sua contabilità secondo la divisione calcolata. Sono due ruoli distinti dentro la stessa transazione. Se il callback fallisce, il revert annulla anche il trasferimento precedente e gli altri effetti di quella transazione.

12 / RESTITUIRE IL CAPITALE

Chi recupera per primo

Il capitale segue una priorità; il gain segue una proporzione.

Se Alice ha finanziato 6 ETH e Carlo 4, il capitale torna prima ad Alice. Quando rientrano 2 ETH di *capitale*, il credito di Alice scende da 6 a 4, quello di Carlo resta 4. La priorità è data dalla quota iniziale, non viene ricalcolata cambiando ordine dopo ogni rata.

contracts/FundingPool.sol / righe 207–224

```
207         for (uint256 i = 0; i < order.length && remaining > 0; i++) {
208             address contributor = order[i];
209             uint256 lockedForLoan = loanContributions[proposalId][contributor];
210             if (lockedForLoan == 0) continue;
211 
212             uint256 refund = remaining < lockedForLoan ? remaining : lockedForLoan;
213 
214             loanContributions[proposalId][contributor] -= refund;
215             lockedContributions[contributor] -= refund;
216 
217             totalUnlockedFunds += refund;
218             remaining -= refund;
219             emit FundsUnlocked(contributor, refund);
220         }
221 
222         if (remaining > 0) {
223             compensationPoolBalance += remaining;
224         }
```

Il ciclo usa un elenco già ordinato. `remaining` è il capitale ancora da assegnare; `lockedForLoan` è il credito residuo di questa persona su questo prestito. Il valore `refund` prende il minore dei due. Se restano 3 da assegnare e Alice ha credito 1, ad Alice va 1 e il ciclo continua con gli altri 2.

Le righe 214–215 riducono sia il credito su quel prestito sia il bloccato totale della persona. La riga 217 aumenta il disponibile del pool. Non aumentiamo `totalContributed`: il capitale era già contato. Un eventuale capitale non più assegnabile, ad esempio perché un credito è già stato compensato, finisce nella riserva.

contracts/ContributorOrder.sol / righe 30–33

```
30     function _before(Position memory a, Position memory b) private pure returns (bool) {
31         return a.initialShare > b.initialShare
32             || (a.initialShare == b.initialShare && uint160(a.account) >= uint160(b.account));
33     }
```

Questa funzione confronta due posizioni: prima viene la quota iniziale maggiore; in parità, l'indirizzo numericamente maggiore. `uint160` permette il confronto numerico degli indirizzi. Il verso decrescente è la scelta documentata per rendere deterministico lo spareggio.

> **Il professore potrebbe chiedere: a cosa serve loanOrderPrepared?**
>
> A non rifare l'ordinamento a ogni rata. Un merge sort prepara la priorità una volta per prestito; il flag ricorda che il lavoro è già fatto. La preparazione ritardata serve anche ai prestiti preesistenti che non avevano ancora quel flag impostato.

13 / GLI INTERESSI

Accreditare non è inviare denaro

Il gain diventa un credito separato, poi Alice lo incassa.

Con quote iniziali 6 e 4, Alice ha diritto al 60% del gain e Carlo al 40%. Se il gain da dividere è 0,025 ETH, riceveranno crediti di 0,015 e 0,010. Anche se Alice ha recuperato prima il capitale, la sua quota iniziale rimane quella usata per gli interessi.

contracts/FundingPool.sol / righe 304–313

```
304             uint256 share = Math.mulDiv(gainAmount, initialLocked, totalInitialLocked);
305             if (share > 0) {
306                 pendingInterestGain[contributor] += share;
307                 disbursed += share;
308             }
309         }
310
311         uint256 leftover = gainAmount - disbursed;
312         if (leftover > 0) {
313             compensationPoolBalance += leftover;
```

Queste righe sono dentro il ciclo sui finanziatori. Ogni quota viene aggiunta a `pendingInterestGain`: è un aggiornamento del registro, non un bonifico. `disbursed` somma ciò che è stato accreditato, nonostante il nome possa far pensare a denaro già inviato. Le divisioni intere possono lasciare qualche wei: il resto viene aggiunto alla riserva.

contracts/FundingPool.sol / righe 330–338

```
330     function claimInterestGain() external {
331         uint256 amount = pendingInterestGain[msg.sender];
332         if (amount == 0) revert InvalidState();
333
334         pendingInterestGain[msg.sender] = 0;
335
336         (bool sent, ) = payable(msg.sender).call{value: amount}("");
337         if (!sent) revert TransferFailed();
338         emit InterestClaimed(msg.sender, amount);
```

Qui invece avviene l'invio. Si legge il credito del chiamante, lo si azzera prima della chiamata esterna e si trasferisce l'importo al suo wallet. Se il trasferimento fallisce, il revert ripristina anche il credito.

Perché separare i due momenti?

Durante il rimborso non inviamo ETH a tutti i contributor uno per uno. Ognuno ritira il proprio gain con una chiamata esplicita: è il modello *pull*, cioè "vengo a prendere ciò che mi spetta". Si separa il guadagno dal capitale prestabile e non si delega a ogni destinatario l'esito della distribuzione durante una rata.

> **Il professore potrebbe chiedere: perché azzeri pendingInterestGain prima di inviare ETH?**
>
> Per impedire che un contratto destinatario rientri e incassi di nuovo lo stesso credito. Se richiama la funzione, il credito è già zero. Se invece l'invio fallisce, l'intera operazione viene annullata e il credito non è perso.

14 / IL FALLIMENTO

Una scadenza non esegue codice

Il default viene registrato quando qualcuno manda una transazione valida.

Supponiamo che il prestito scada al blocco 100. Un rimborso completo nel blocco 100 è ancora puntuale. Dal 101 è tardivo. Il nodo non chiama automaticamente una funzione allo scoccare del blocco: finché non avviene una transazione, lo stato salvato può ancora dire Funded.

contracts/LoanContract.sol / righe 150–156

```
150     function markDefaulted() external {
151         if (status != LoanStatus.Funded) revert InvalidState();
152         if (block.number <= dueBlock) revert LoanNotOverdue();
153 
154         status = LoanStatus.Defaulted;
155         IFundingPool(pool).handleLoanDefaulted();
156         emit Defaulted(dueBlock);
```

Questa funzione verifica che il prestito sia ancora finanziato e che il blocco di scadenza sia stato superato. Poi aggiorna lo stato del prestito, avvisa il pool e produce un evento. Non è riservata al borrower: i controlli sullo stato e sul tempo impediscono di dichiarare fallito un prestito non ancora scaduto.

contracts/FundingPool.sol / righe 423–435

```
423     function handleLoanDefaulted() external {
424         if (!authorizedLoanContracts[msg.sender]) revert Unauthorized();
425         uint256 proposalId = loanToProposalId[msg.sender];
426         if (!isLoanFailed[proposalId]) {
427             isLoanFailed[proposalId] = true;
428             failedLoanCount += 1;
429             if (!loanResolved[msg.sender]) {
430                 loanResolved[msg.sender] = true;
431                 activeLoanCount -= 1;
432             }
433             _adjustCollateralPercentage(false);
434             emit LoanDefaulted(proposalId, msg.sender);
435         }
```

Dentro questo callback `msg.sender` è il contratto del prestito. Il pool verifica che sia autorizzato. `isLoanFailed` impedisce di aumentare più volte il contatore dei fallimenti; `loanResolved` impedisce di scalare più volte quello degli attivi. Alla prima registrazione si aggiustano anche le percentuali: fallimento $+5$ punti, successo -5, nei limiti 1–100%.

Un rimborso tardivo o una richiesta di compensazione possono anch'essi registrare il default. Il prestito fallito accetta ancora pagamenti, ma non torna Repaid: il denaro recuperato dopo non cancella il fatto che Bob fosse in ritardo.

> **Il professore potrebbe chiedere: un prestito risolto è per forza tutto rimborsato?**
>
> No. Risolto significa che abbiamo registrato l'esito, anche fallimento. Può non esserci più fra gli attivi e avere ancora capitale non recuperato. Per questo il contatore degli attivi non basta per sapere quanti ETH Alice può ritirare.

15 / LA COMPENSAZIONE

Recuperare solo ciò che c'è

La riserva aiuta a coprire una perdita, ma non la garantisce per intero.

Nella demo Bob non restituisce un prestito di 0,5 ETH. La riserva contiene solo 0,025 ETH, prodotti dal primo prestito andato bene. Prima dell'estratto, la funzione controlla prestito, scadenza e credito della persona e, se serve, registra il default.

contracts/FundingPool.sol / righe 361–383

```
361         uint256 actualCompensation = owedAmount;
362         if (actualCompensation > compensationPoolBalance) {
363             actualCompensation = compensationPoolBalance;
364         }
365 
366         // A valid claim still records the default when no compensation is
367         // available. Keep the claim intact so it can be retried after a refill.
368         if (actualCompensation == 0) {
369             emit CompensationRequested(msg.sender, 0);
370             return;
371         }
372 
373         loanContributions[proposalId][msg.sender] -= actualCompensation;
374         lockedContributions[msg.sender] -= actualCompensation;
375         // Compensation is paid out of the compensation pool, not converted
376         // into a second withdrawable claim against the funding pool.
377         totalContributed[msg.sender] -= actualCompensation;
378         _unregisterEmptyContributor(msg.sender);
379 
380         compensationPoolBalance -= actualCompensation;
381 
382         (bool sent, ) = payable(msg.sender).call{value: actualCompensation}("");
383         if (!sent) revert TransferFailed();
```

All'inizio si limita il pagamento al minore tra credito residuo e riserva. Se il risultato è zero, si emette comunque l'evento e si termina senza ridurre il credito: Alice potrà riprovare se la riserva viene alimentata di nuovo.

Quando vengono pagati 0,025, diminuiscono *sia* il credito bloccato *sia* il totale di Alice. Con 50 totali e 0,5 bloccati, il disponibile è 49,5. Dopo il pagamento resta $49{,}975 - 0{,}475 = 49{,}5$. Il disponibile non sale: gli ETH sono già stati inviati al wallet.

> **Il professore potrebbe chiedere: perché sottrai anche da totalContributed?**
>
> Perché sto pagando fuori dal pool, non rendendo capitale ritirabile una seconda volta. Se riducessi solo il bloccato, Alice riceverebbe 0,025 nel wallet e vedrebbe salire di altri 0,025 il saldo da ritirare. Sarebbe un doppio beneficio per lo stesso pagamento.

16 / ORACLE SU ETHEREUM

Una richiesta e una risposta separata

Il contratto registra il saldo, ma non legge da solo la rete Bitcoin.

contracts/BitcoinOracle.sol / righe 33–38

```
33      function requestUpdate(string calldata btcAddress) external payable {
34          if (msg.value < minFee) revert InsufficientFee();
35
36          // Emits the event that will trigger the Python script
37          emit UpdateRequested(btcAddress, msg.sender, msg.value);
38      }
```

Bob paga almeno `minFee` e viene emesso `UpdateRequested`. Qui non viene ancora scritto alcun saldo. L'evento è una richiesta che il servizio Python deve osservare e servire: pagare non significa ricevere una risposta sincrona nella stessa chiamata.

contracts/BitcoinOracle.sol / righe 45–48

```
45      function update(string calldata btcAddress, uint256 balanceInSatoshi) external onlyOwner {
46          balances[btcAddress] = balanceInSatoshi;
47          emit BalanceUpdated(btcAddress, balanceInSatoshi);
48      }
```

Questo è il secondo momento. Il servizio firma con la chiave dell'operatore; `onlyOwner` limita chi può aggiornare il registro. Il numero è in satoshi, non in BTC decimali e non in wei. Dopo la scrittura un evento segnala la risposta.

contracts/BitcoinOracle.sol / righe 54–56

```
54      function getBalance(string calldata btcAddress) external view returns (uint256) {
55          return balances[btcAddress];
56      }
```

Il pool legge il valore memorizzato. Un mapping non valorizzato restituisce zero: dalla sola lettura zero non posso distinguere indirizzo vuoto da indirizzo mai aggiornato. La demo aspetta quindi un evento nuovo dopo la propria richiesta. Anche un evento con saldo zero è una risposta valida, ma non dà capacità di prestito.

Il costo e il limite di fiducia

La fee minima è calibrata sul nostro build: 49.426 gas per 0,1 gwei, cioè 0,0000049426 ETH. Gas e prezzo del gas sono due quantità diverse. L'operatore è fidato: il controllo dell'owner evita scritture non autorizzate, ma non prova che l'operatore stia dicendo il vero.

> **Il professore potrebbe chiedere: quindi i Bitcoin vengono bloccati a garanzia?**
>
> No. Verifichiamo un saldo storico, ma non proviamo che Bob controlli quell'indirizzo e non blocchiamo i BTC. È il modello semplificato del progetto, non una garanzia che il contratto possa sequestrare in caso di default. Il cambio è fissato a 30 ETH per BTC.

17 / IL SALDO BITCOIN

Contare soltanto gli output non spesi

Un esempio concreto prima dei dizionari Python.

Immagina che Alice riceva un output da 2 BTC. Poi lo spenda per pagare Bob 0,4 BTC, ricevendo 1,6 BTC di resto, trascurando la commissione nell'esempio. Non possiede $2 + 1{,}6$: il vecchio output da 2 è stato consumato. Ora possiede solo il nuovo output da 1,6. Un *UTXO* è proprio un output non ancora speso.

python/oracle/oracle_service.py / righe 94–106

```
94          if not is_coinbase(tx):
95              for vin in tx.vin:
96                  spent_txid = b2lx(vin.prevout.hash)
97                  spent_vout = vin.prevout.n
98                  changes.append(((spent_txid, spent_vout), None))
99
100         for i, vout in enumerate(tx.vout):
101             address = script_to_address(vout.scriptPubKey)
102             if address is None:
103                 continue
104             if vout.nValue < 0:
105                 raise BlockDataError(f"Negative output value at height {block_height}")
106             changes.append(((txid, i), (address, vout.nValue)))
```

Per una transazione ordinaria, gli input indicano vecchi output da eliminare. La coppia `spent_txid, spent_vout` identifica precisamente quale output è stato speso. Poi il ciclo sugli output registra i nuovi importi e gli indirizzi riconoscibili dal parser. `None` nella lista delle modifiche significa "elimina questo output", non "il suo valore è zero".

La coinbase viene trattata separatamente perché crea nuovi output senza spendere un output precedente ordinario. La ricompensa originale del genesis è esclusa: non fa parte degli output spendibili. Eventuali donazioni successive allo stesso indirizzo, invece, possono comparire nel saldo.

python/oracle/oracle_service.py / righe 124–125

```
124 def get_address_balance(btc_address: str) -> int:
125     return sum(addr_utxos.get(btc_address, {}).values())
```

`addr_utxos` raccoglie gli output ancora presenti per indirizzo. La funzione somma i loro valori. Se l'indirizzo non è presente, `get` restituisce il dizionario vuoto e la somma vale zero.

Perché preparare prima le modifiche del blocco?

La funzione costruisce la lista di rimozioni e inserimenti prima di applicarla. Se c'è un errore, non vogliamo lasciare mezzo blocco nei registri. Il codice conserva i vecchi valori dei soli output toccati per poter annullare l'applicazione, senza copiare ogni volta l'intero insieme UTXO.

> **Il professore potrebbe chiedere: perché non sommi tutte le transazioni ricevute dall'indirizzo?**
>
> Perché contarei anche denaro che è già stato speso. Per sapere il saldo devo ricostruire gli output ancora utilizzabili dopo aver applicato entrate e spese in ordine.

18 / L'ATTESA DELL'ORACLE

Che cosa significa ORACLE READY

Non è una pausa scelta a caso: è la fine della ricostruzione dello snapshot.

All'avvio l'oracle legge i file Bitcoin locali. Il parser sceglie la catena collegata al genesis con il lavoro cumulativo maggiore e applica in ordine i primi **131.000 blocchi**, dalle altezze 0 a 130.999. Controlla anche hash e Merkle root, ma non implementa tutte le regole di consenso di un nodo Bitcoin.

python/oracle/oracle_service.py / righe 134–144

```
134         for block_height, block in parse_blocks(blk_dat_dir, max_blocks=MAX_BLOCKS):
135             if block_height != blocks_processed or blocks_processed >= MAX_BLOCKS:
136                 raise BlockDataError(f"Unexpected Bitcoin block height {block_height}; expected {
        blocks_processed}")
137             process_block(block, block_height)
138             blocks_processed += 1
139             if blocks_processed % 10_000 == 0:
140                 logger.info("  Processed %6d blocks | UTXO set: %d entries", blocks_processed, len(utxo_set))
141         if blocks_processed != MAX_BLOCKS:
142             raise BlockDataError(
143                 f"Incomplete Bitcoin snapshot: processed {blocks_processed}/{MAX_BLOCKS} blocks"
144             )
```

Il ciclo pretende l'altezza successiva a quella già elaborata. Dopo ogni blocco incrementa il contatore; alla fine rifiuta una sequenza troppo corta. Non salta un blocco danneggiato per continuare pubblicando saldi incompleti. Se la sincronizzazione fallisce, gli indici vengono svuotati e il servizio non si dichiara pronto.

python/oracle/oracle_service.py / righe 195–201

```
195     database = os.getenv("EVENT_DB_PATH", str(Path(__file__).with_name(f"events-{oracle_addr.lower()}.sqlite3
        ")))
196     store = EventStore(database, store_identity(w3, oracle_addr, operator_account.address),
197                        int(os.getenv("DEPLOYMENT_BLOCK", "0")))
198     try:
199         sync_bitcoin_blocks(blk_dat_dir)
200         logger.info("ORACLE READY: Bitcoin sync complete. You can now run simulate.py.")
201         run_polling_loop(oracle_contract, operator_account, w3, store)
```

La coda degli eventi viene aperta prima della lettura lunga. Poi si ricostruisce Bitcoin, si stampa `ORACLE READY` e si cominciano a servire le richieste. Partire dal blocco di deployment evita di dimenticare richieste arrivate mentre il programma era ancora occupato con Bitcoin.

Due memorie diverse, da non confondere

Il saldo Bitcoin viene ricostruito in RAM: a ogni riavvio ricomincia il replay. SQLite conserva invece eventi Ethereum e transazioni da inviare, non una cache dei saldi BTC. Una volta pronto, l'oracle risponde usando l'indice in memoria, senza rileggere tutti i blocchi per ogni richiesta.

> **Il professore potrebbe chiedere: un aggiornamento fresco contiene il saldo BTC di oggi?**
>
> No. È una nuova risposta ottenuta dallo snapshot storico richiesto. "Fresco" nella demo significa successivo alla nostra richiesta Ethereum, non aggiornato all'ultimo blocco Bitcoin mondiale. Per questo aspetto READY, non un numero fisso di secondi.

19 / IL VOTER AUTOMATICO

Chi manda davvero il voto YES

Python esegue una strategia; il contratto applica le regole.

Il voter legge gli eventi `LoanProposed`. Per ogni proposta cerca di registrare YES con l'account configurato. Non assegna direttamente il prestito e non salta i controlli di contributo, voto unico e risoluzione.

python/voter/auto_voter.py / righe 27–42

```
27 def evaluate_and_vote(pool_contract, voter_account, w3, proposal_id, store, event_id):
28     if pool_contract.functions.hasVoted(proposal_id, voter_account.address).call():
29         if not pool_contract.functions.voteSupport(proposal_id, voter_account.address).call():
30             raise PermanentEventError(f"Proposal {proposal_id} already has a NO from this account; votes
       cannot be changed")
31         logger.info("Proposal %s already has this contributor's vote", proposal_id)
32         return
33     proposal = pool_contract.functions.proposals(proposal_id).call()
34     if proposal[8] or pool_contract.functions.terminated().call():
35         raise PermanentEventError(f"Proposal {proposal_id} or the service is already closed; vote missed")
36     # Contributors with fully locked funds still cast a YES with zero current
37     # weight. The contract recomputes the weight when the applicant resolves.
38     if pool_contract.functions.totalContributed(voter_account.address).call() == 0:
39         raise RuntimeError("Voter has no pool contribution yet; fund its position and retry")
40     receipt = send_durable(
41         w3, voter_account, pool_contract.functions.voteOnProposal(proposal_id, True), store, event_id,
42     )
```

Se l'account ha già votato YES, non viene mandata un'altra transazione. Se aveva votato NO, il programma segnala un errore definitivo: non può cambiare un voto già registrato. Se la proposta o il servizio sono chiusi, non insiste come se fosse un problema temporaneo di rete.

Il controllo sul contributo usa il totale, non il disponibile. Se Alice ha 10 ETH interamente bloccati, può ancora votare con peso corrente zero. Se prima della risoluzione un altro prestito le restituisce capitale, il suo peso potrà diventare positivo. Questa regola deve essere rispettata sia da Solidity sia dal programma Python.

Perché il programma legge prima di scrivere?

Per capire se la richiesta è ancora da eseguire, se è già stata soddisfatta o se ormai non può più esserlo. Un'interruzione può avvenire dopo che il voto è arrivato on-chain ma prima che il programma abbia salvato l'esito locale. Ricontrollare lo stato evita di trattare quel caso come un nuovo voto.

> **Il professore potrebbe chiedere: dove si vede che la strategia è sempre YES?**
>
> Nella riga 41: `voteOnProposal(proposal_id, True)`. Il booleano passato è vero. I controlli prima non scelgono una proposta conveniente: verificano solo che il voto possa ancora essere registrato correttamente. La maggioranza resta calcolata dal contratto, usando tutti i contributor.

20 / EVENTI E RIAVVII

Come evitiamo di perdere una richiesta

Pensala come una lista di lavori da fare e di risposte già preparate.

Un servizio non deve fidarsi soltanto della memoria del processo: se si chiude, quella memoria sparisce. SQLite conserva quali eventi sono arrivati, quali restano da servire e quali transazioni sono già state preparate.

python/durable_events.py / righe 52–62

```
52      def enqueue_scan(self, events, through_block, block_hash=None):
53          """Durably enqueue *all* decoded logs before moving the scan cursor."""
54          with self.db:
55              for event_id, block_number, payload in events:
56                  self.db.execute(
57                      "INSERT OR IGNORE INTO events(id, block_number, payload) VALUES (?, ?, ?)",
58                      (event_id, block_number, json.dumps(payload)),
59                  )
60              self.db.execute("UPDATE metadata SET value=? WHERE key='next_block'", (str(through_block + 1),))
61              if block_hash is not None:
62                  self.db.execute("INSERT OR REPLACE INTO metadata VALUES ('checkpoint_hash', ?)", (block_hash
        ,))
```

`with self.db` raggruppa inserimento degli eventi e aggiornamento del cursore in una transazione SQLite. Il cursore indica da quale blocco cercare la prossima volta. Non devo avanzarlo senza aver salvato i lavori: dopo un crash salterei richieste che non ho mai servito. `INSERT OR IGNORE` evita il doppione di un evento già presente con lo stesso identificativo.

python/durable_events.py / righe 173–177

```
173          raw_tx = bytes(account.sign_transaction(transaction).raw_transaction)
174          tx_hash = w3.to_hex(w3.keccak(raw_tx))
175          store.save_transaction(event_id, raw_tx.hex(), tx_hash)
176      try:
177          w3.eth.send_raw_transaction(raw_tx)
```

Qui la risposta viene firmata e salvata *prima* dell'invio. Se cade la connessione dopo la trasmissione, al riavvio posso cercare la stessa transazione e, se necessario, ritrasmettere gli stessi byte. Non creo alla cieca una risposta nuova con un altro nonce, che potrebbe pagare due volte la stessa operazione.

Nonce e hash, senza confonderli

Il nonce ordina le transazioni di un account. L'hash identifica quella precisa transazione firmata. Gli account dei servizi devono essere dedicati: se un altro programma ne usa i nonce, possiamo incontrare conflitti da ispezionare. La ricevuta dice poi se la transazione è stata inclusa ed è riuscita.

> **Il professore potrebbe chiedere: SQLite garantisce che tutto avvenga esattamente una volta in ogni rete?**
>
> No. Rende affidabili i retry nel contesto della catena privata prevista. I checkpoint di hash segnalano reset o riorganizzazioni; non c'è una procedura automatica generale di recupero dalle reorg. Di fronte a un guasto RPC non cancello la coda: controllo prima lo stato del nodo e della transazione salvata.

21 / SETUP E TRANSAZIONI

Dal programma Python alla blockchain

Creare un account, firmare e aspettare una ricevuta sono passaggi distinti.

`setup.sh` prepara gli strumenti e compila; `python/setup.py` crea gli account e distribuisce i contratti. Il setup genera deployer, contributor e borrower; l'account prefunded fornito dal corso li finanzia soltanto.

python/setup.py / righe 57–67

```
57      deployer, contributor, borrower = (w3.eth.account.create() for _ in range(3))
58      accounts = {"Deployer": deployer, "Contributor": contributor, "Borrower": borrower}
59      for role, account in accounts.items():
60          transaction = {
61              "to": account.address, "value": w3.to_wei(100, "ether"),
62              "nonce": w3.eth.get_transaction_count(prefunded.address, "pending"),
63              "gas": 21_000, "gasPrice": w3.eth.gas_price, "chainId": 202526,
64          }
65          signed = prefunded.sign_transaction(transaction)
66          tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
67          checked_receipt(w3.eth.wait_for_transaction_receipt(tx_hash))
```

Ogni account riceve 100 ETH. Il prefunded firma questi trasferimenti, ma non è l'account che distribuisce oracle e pool. Il nuovo deployer crea oracle, implementazione e proxy, poi lo script verifica owner e configurazione. I file `.env` locali conservano le chiavi generate: non vanno consegnati né mostrati in una schermata d'esame.

python/simulate.py / righe 27–35

```
27  def send_tx_with_value(w3, account, tx_call, value=0):
28      transaction = tx_call.build_transaction({
29          "from": account.address,
30          "nonce": w3.eth.get_transaction_count(account.address, "pending"),
31          "value": value, "gasPrice": w3.eth.gas_price, "chainId": 202526,
32      })
33      signed = account.sign_transaction(transaction)
34      tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)
35      return checked_receipt(w3.eth.wait_for_transaction_receipt(tx_hash))
```

`build_transaction` costruisce i dati per chiamare una funzione del contratto. `value` è l'importo da inviare; il nonce usa anche le transazioni pendenti; il chain ID è 202526, quello del corso. L'account firma localmente. Il nodo riceve i byte firmati e il client aspetta la ricevuta, controllandone lo stato.

Un hash restituito dal nodo non basta a dire "operazione riuscita". La transazione potrebbe essere ancora pendente o essere inclusa con revert. L'ABI serve al client per codificare nomi, argomenti ed eventi del contratto; il bytecode serve invece quando si distribuisce un nuovo contratto.

> **Il professore potrebbe chiedere: perché non chiami tutto con .call()?**
>
> Perché una lettura `.call()` non salva una modifica sulla blockchain. Per depositare, votare o rimborsare serve una transazione firmata e inclusa in un blocco. Leggere il saldo con `.call()` è diverso dalla `call` Solidity che può inviare ETH.

22 / PROXY E UPGRADE

Cambiare la logica senza cambiare indirizzo

Lo stato del pool rimane nel proxy; i prestiti già creati restano autonomi.

Immagina che il proxy sia l'indirizzo della cassa e l'implementazione sia il manuale di istruzioni. Cambiare manuale non crea una nuova cassa né sposta automaticamente i fondi. Il proxy delega l'esecuzione mantenendo il proprio storage e saldo: è l'idea della *delegatecall*. Anche `msg.sender` rimane quello della chiamata al proxy: non è una normale chiamata che parte dal proxy verso un altro contratto.

contracts/FundingPoolProxy.sol / righe 9–12

```
9  contract FundingPoolProxy is ERC1967Proxy {
10     constructor(address implementation, address oracleAddress)
11         ERC1967Proxy(implementation, abi.encodeCall(FundingPool.initialize, (oracleAddress)))
12     {}
```

Il costruttore riceve l'implementazione e include già la chiamata a `initialize`. Così il proxy nasce inizializzato: non rimane una finestra in cui un estraneo può chiamare per primo e diventare owner. `ERC1967Proxy` usa uno slot standard per conservare l'indirizzo dell'implementazione, separandolo dai dati applicativi.

contracts/FundingPool.sol / righe 114–127

```
114     constructor() {
115         _disableInitializers();
116     }
117
118     function initialize(address oracleAddress) public initializer {
119         if (oracleAddress == address(0)) revert InvalidAddress();
120
121         __Ownable_init(msg.sender);
122         bitcoinOracle = BitcoinOracle(oracleAddress);
123         terminated = false;
124         collateralPercentage = 50;
125     }
126
127     function _authorizeUpgrade(address newImplementation) internal override onlyOwner {}
```

Il costruttore blocca l'inizializzazione dell'istanza di implementazione. Il proxy inizializza invece il proprio stato tramite la logica: imposta owner, oracle e collateral iniziale al 50%. `initializer` evita di rifarlo. L'override alla riga 127 riserva l'upgrade all'owner, sfruttando il meccanismo UUPS.

Che cosa devo preservare durante un upgrade?

Il significato e la disposizione dello storage esistente. Se cambio l'ordine delle variabili senza una migrazione compatibile, la nuova logica potrebbe leggere vecchi valori come se fossero altri dati. I test V2 controllano alcuni casi concreti; non rendono sicuro qualunque aggiornamento futuro.

> **Il professore potrebbe chiedere: se aggiorni il pool, cambia anche il codice dei LoanContract?**
>
> No. Ogni prestito è un contratto autonomo, non un proxy di questo pool. Il suo bytecode e i termini fissati restano quelli iniziali. Anche per questo una nuova logica del pool deve continuare a gestire i callback dei vecchi prestiti.

23 / REENTRANCY

Perché il destinatario può richiamarci

Un trasferimento a un contratto può far eseguire altro codice.

Il pool normale riduce i registri prima di trasferire ETH. Per mostrare il rischio, nei test esiste anche una variante volutamente errata: invia prima e aggiorna dopo. Non viene distribuita dal setup del servizio.

contracts/test/FundingPoolVulnerable.sol / righe 14–23

```
14          uint256 positionBefore = totalContributed[msg.sender];
15          uint256 availableBefore = totalUnlockedFunds;
16          (bool sent, ) = payable(msg.sender).call{value: amount}("");
17          if (!sent) revert TransferFailed();
18
19          // Intentionally broken: stale assignments AFTER the external call.
20          // A checked subtraction of the live balance could instead revert and
21          // undo the attack in Solidity 0.8; this variant demonstrates real theft.
22          totalContributed[msg.sender] = positionBefore - amount;
23          totalUnlockedFunds = availableBefore - amount;
```

Durante la chiamata esterna, il saldo non è ancora stato ridotto. Il contratto attaccante riceve ETH e richiama `withdraw()`; il pool vede ancora il credito precedente e invia di nuovo. Alla fine vengono riscritti i valori salvati prima della ricorsione, ormai vecchi.

contracts/test/ReentrancyAttacker.sol / righe 30–37

```
30      receive() external payable {
31          if (withdrawalAmount > 0 && address(victim).balance >= withdrawalAmount) {
32              // A protected pool rejects the recursive withdrawal. Catch that
33              // failure so the initial legitimate withdrawal can still complete.
34              (bool success, ) = address(victim).call(
35                  abi.encodeCall(IWithdrawablePool.withdraw, (withdrawalAmount))
36              );
37              if (success) successfulReentries += 1;
```

La funzione `receive()` viene eseguita quando arrivano ETH senza dati. Se il pool ha ancora denaro, l'attaccante tenta un altro prelievo. La chiamata a basso livello restituisce un booleano: se il pool normale rifiuta la ricorsione, l'attaccante può lasciar terminare il prelievo iniziale legittimo.

Nei test l'onesto mette 10 ETH e l'attaccante 1. La variante vulnerabile può essere svuotata anche dei 10 altrui. Nel pool normale la ricorsione fallisce e rimangono i fondi dell'onesto.

> **Il professore potrebbe chiedere: bastava spostare una sottrazione dopo la call?**
>
> Non necessariamente. In Solidity 0.8 una sottrazione sul saldo corrente può andare in underflow dopo una chiamata ricorsiva e annullare tutto l'attacco. La nostra variante usa assegnazioni con valori vecchi per mostrare un furto effettivo. Spiego questo codice preciso, non sostengo che ogni variante apparentemente simile sia sfruttabile allo stesso modo.

24 / TEST E GAS

Come leggere una prova

Un test prepara una situazione, esegue un'azione e controlla il risultato.

`test/trace_compliance.test.ts` / righe 20–31

```
20  it("deducts proportional rounding from the actual loan principal", async () => {
21    const f = await deployProtocol();
22    const third = f.wallets[4];
23    const deposits = [parseEther("2"), parseEther("3"), parseEther("6")];
24    for (const [index, wallet] of [f.contributor, f.smallContributor, third].entries()) {
25      await f.receipt(await f.pool.write.contribute({ account: wallet.account, value: deposits[index] }));
26    }
27    const requested = parseEther("4");
28    const { loan } = await f.approveLoan(requested, { voters: [f.contributor, f.smallContributor, third] });
29    assert.ok(await loan.read.principal() < requested);
30    assert.equal(await loan.read.principal(), deposits.reduce((sum, deposit) => sum + deposit * requested /
       parseEther("11"), 0n));
31  });
```

`deployProtocol()` prepara contratti e account isolati. Tre persone depositano 2, 3 e 6 ETH; poi viene richiesto un prestito di 4. `await` aspetta il completamento delle operazioni asincrone del test. Le ultime due righe controllano che il capitale sia inferiore alla richiesta per arrotondamento e coincida con la somma delle quote calcolate per difetto. `BigInt`, riconoscibile dal suffisso `n`, permette conti interi.

`test/gas_profiling.test.ts` / righe 12–16

```
12    async function measure(name: string, hash: Hash) {
13      const r = await f.receipt(hash);
14      operations[name] = r.gasUsed.toString();
15      assert.ok(r.gasUsed > 0n);
16    }
```

Questo helper aspetta la ricevuta e registra `gasUsed`, il gas consumato. Non usa il limite massimo di gas inviato con la transazione. Il costo in wei si ottiene moltiplicando il gas consumato per il prezzo del gas. Per esempio 49.426 gas per 0,1 gwei fanno 4.942.600.000.000 wei.

I risultati riportati nel progetto sono 34 test contratti, di cui 4 Solidity e 30 TypeScript, e 49 test Python offline. Uno scenario Solidity esegue 256 casi fuzz, cioè varia gli input per controllare una proprietà. Non sono 256 ulteriori test distinti. Il profiling copre 20 scenari.

Che cosa non dimostra un singolo test

Il test isolato può minare blocchi immediatamente; la demo Geth no. Un mock Python non prova che tutto il dataset reale venga letto correttamente. Il gas con un contributor non è una stima universale per mille contributor. Il target del compilatore è Berlin, compatibile con il genesis del corso.

> **Il professore potrebbe chiedere: dove trovo la prova di una regola specifica?**
>
> Cerco il test per scenario: `trace_compliance.test.ts` per i comportamenti di traccia, `requirements_boundaries.test.ts` per i limiti, `reentrancy.test.ts` per l'attacco e `upgrade.test.ts` per il proxy. Posso spiegare preparazione, azione e controllo, non solo dire "i test passano".

25 / REGOLE ED ECONOMIA

Un comportamento scorretto senza bug

Rispettare il codice non rende ogni incentivo automaticamente sicuro.

Supponiamo che una persona malevola metta 6 ETH e una persona onesta 4. La prima controlla la maggioranza e si accorda con il borrower. Può far approvare un prestito di 10 ETH, purché superi i controlli, e il gruppo può poi non rimborsarlo.

Il gruppo riceve 10, ma 6 erano già capitale proprio: il guadagno netto ai danni dell'onesto è 4, trascurando gas e altri flussi. Dire che guadagna 10 sarebbe contare come profitto anche denaro che possedeva prima.

Il dato BTC non impedisce questo comportamento: non richiediamo prova del possesso dell'indirizzo né blocchiamo i Bitcoin. Inoltre la priorità di recupero del capitale favorisce chi aveva la quota iniziale più grande. Sono conseguenze delle regole economiche, non un attacco di reentrancy.

Perché non cambiare subito il sistema di voto?

Si potrebbero immaginare altre garanzie, ma il progetto segue la traccia. Cambiare il peso corrente in uno snapshot di voto, introdurre un vero vincolo su Bitcoin o redistribuire il rischio in altro modo significherebbe cambiare il protocollo. All'orale distinguo ciò che abbiamo implementato da possibili evoluzioni, senza attribuire al progetto protezioni che non possiede.

contracts/FundingPool.sol / righe 133–137

```
133     function terminate() external onlyOwner {
134         if (activeLoanCount != 0) revert ActiveLoansExist();
135         terminated = true;
136         emit Terminated();
137     }
```

Questa è la terminazione amministrativa. Solo l'owner può attivarla e non devono esserci prestiti attivi. `terminated` blocca il nuovo business, non distrugge il contratto. Restano le operazioni previste di prelievo, incasso e recupero. Un default risolto può comunque lasciare un credito scoperto.

> **Il professore potrebbe chiedere: se terminate vale true, gli ETH spariscono?**
>
> No. È un flag che alcune funzioni controllano tramite `notTerminated`. Non è un selfdestruct e non rende liquido il capitale non recuperato. Devo guardare quali funzioni usano il modifier, non immaginare che il flag spenga ogni singola operazione.

26 / LA DEMO CONCRETA

Prima il successo, poi il default

Collegare i messaggi sul terminale alle funzioni già studiate.

La demo si avvia con oracle e voter già in esecuzione. Prima aspetta una risposta BTC nuova, poi finanzia la posizione del contributor. Nel percorso standard, su deployment nuovo, il deposito è 50 ETH e il primo prestito è 1 ETH al 5%. Due rate da 0,525 lo chiudono: gain e riserva ricevono 0,025 ciascuno. Il gain viene incassato e il collateral globale scende da 50 a 45%.

`python/simulate.py` / righe 268–280

```
268     print("7. Scenario B: creating a second loan that will not be repaid...")
269     failed_id, failed_loan = create_demo_loan(
270         w3, pool, accounts, args.btc_address, max(1, principal // 2),
271         rate=10, duration=3, timeout=args.timeout,
272     )
273     print("8. Waiting for real chain blocks until the second loan is overdue...")
274     wait_for_expiry(w3, failed_loan, args.timeout)
275     print("The deadline has passed. Storage changes only when a transaction records the default.")
276     print_balances(w3, pool, accounts)
277     print_loan_state(w3, pool, failed_loan, failed_id, contributor)
278 
279     print("9. Requesting compensation; this transaction also records the default...")
280     claim_demo_compensation(w3, pool, failed_loan, failed_id, contributor)
```

Il secondo prestito usa metà del capitale richiesto e dura tre blocchi. `max(1, principal // 2)` evita zero per importi minuscoli; `//` è divisione intera Python. `wait_for_expiry` aspetta un blocco strettamente maggiore di `dueBlock`. La richiesta di compensazione registra il default e paga il minore fra credito e riserva. Poi la demo ripete la richiesta per mostrare il comportamento con riserva vuota.

Stato finale del percorso standard	Valore
Primo prestito / secondo prestito	Repaid / Defaulted
Compensazione pagata / secondo tentativo	0,025 / 0 ETH
Posizione totale del contributor	49,975 ETH
Disponibile / bloccato residuo	49,5 / 0,475 ETH
Riserva / gain da incassare	0 / 0 ETH
Prestiti attivi / falliti	0 / 1
Collateral globale	50%

Questi numeri descrivono un deployment nuovo con un contributor. Su uno già usato, altri saldi e riserve possono cambiare i risultati. I wallet pagano anche il gas, quindi non vanno confrontati con somme che lo ignorano. La prova Geth salvata in `docs/default-demo-verification.json` registra 20 ricevute riuscite e 16 controlli finali per i sorgenti identificati dagli hash.

> **Il professore potrebbe chiedere: perché il bloccato non torna a zero alla fine?**
>
> Perché il secondo prestito non è stato rimborsato. Solo 0,025 dei suoi 0,5 ETH sono stati recuperati dalla riserva. I rimanenti 0,475 sono un credito non recuperato, non ETH ritirabili.

27 / PREPARARSI ALLA LETTURA

Se il professore indica una riga

Parti dal contesto, poi spiega l'effetto e il motivo.

Non iniziare traducendo ogni simbolo se non hai ancora detto dove ti trovi. Puoi usare questo ordine: "Questa funzione viene chiamata da…; serve a…; prima controlla…; poi aggiorna…; questa scelta evita…". Un piccolo esempio numerico rende la risposta molto più chiara di un elenco di parole tecniche.

Ti indica: totalContributed - lockedContributions.

"Sto calcolando il capitale libero della persona. Se ha 10 ETH totali e 4 prestati, ne può usare o ritirare 6. Non sto leggendo il saldo del suo wallet."

Ti indica: proposal.noVotes = currentDisposable - currentYesVotes.

"Tutto il capitale disponibile non associato a un YES viene contato come NO. Comprende anche chi non ha votato. I pesi sono quelli attuali alla risoluzione."

Ti indica: initialLoanContributions e loanContributions.

"La prima conserva il contributo iniziale per priorità e interessi; la seconda è il credito residuo, che diminuisce con rimborsi e compensazioni. Non sono due fondi separati."

Ti indica: !loanResolved[msg.sender].

"Sto evitando di trattare due volte la chiusura dello stesso prestito. Il contatore degli attivi non deve scendere a ogni callback successivo. Qui msg.sender è il contratto del prestito."

Ti indica: if (actualCompensation == 0) return.

"Il credito è valido, ma non c'è riserva. Il default è stato registrato prima; esco senza cancellare il credito. In futuro si può riprovare se arrivano fondi nella riserva."

Ti indica: balances[btcAddress] = balanceInSatoshi.

"Salvo in Ethereum la risposta pubblicata dall'operatore. Non ricostruisco Bitcoin in questa riga; quel lavoro è nel programma Python. Solo l'owner può aggiornare questo dato."

Ti indica: store.save_transaction prima di send_raw_transaction.

"Salvo la risposta firmata prima di inviarla. Se il processo si interrompe, posso cercare e ritentare quella stessa transazione senza crearne una nuova alla cieca."

> **L'ultimo ripasso**
>
> Apri a caso una delle funzioni mostrate. Spiega un caso riuscito e uno rifiutato, chi è il chiamante e che cosa cambia nei saldi. Se dici "è per sicurezza", aggiungi quale problema preciso evita. Se dici "lo richiede la traccia", distingui la regola dalla nostra scelta di implementazione. Non occorre ricordare il numero di riga: occorre sapersi orientare nel flusso del prestito.